Extensions en Swift

420-249-AH - Programmation mobile iOS

1. Qu'est-ce qu'une extension ?

Une extension permet d'ajouter de nouvelles fonctionnalités à un type existant.

Swift utilise :

```
extension
```

On peut étendre :

```
class
struct
enum
protocol
Int
String
Double
Array
```

sans modifier directement leur déclaration originale.

2. Exemple simple

```
extension String {
    func afficherMajuscules() {
        print(self.uppercased())
    }
}
```

Utilisation :

```
let nom = "khalil"
nom.afficherMajuscules()
```

Résultat :

```
Console
KHALIL
```

3. Pourquoi utiliser une extension ?

Une extension permet notamment :

- D'organiser le code
- D'ajouter des méthodes

- D'ajouter des propriétés calculées
- D'adopter un protocole
- D'enrichir des types existants

Elle évite aussi de surcharger une seule déclaration avec trop de code.

4. Étendre un type créé par nous

Supposons :

```
class Appareil {
    var nom: String
    var batterie: Int

    init(nom: String, batterie: Int) {
        self.nom = nom
        self.batterie = batterie
    }
}
```

On peut ajouter une fonctionnalité ailleurs :

```
extension Appareil {
    func afficherEtat() {
        print("\(nom) : \(batterie)%")
    }
}
```

Puis :

```
let iphone = Appareil(
    nom: "iPhone",
    batterie: 80
)
iphone.afficherEtat()
```

5. Ajouter une propriété calculée

Une extension peut ajouter une propriété calculée.

```
extension Appareil {
    var batterieFaible: Bool {
        return batterie <= 20
    }
}
```

Utilisation :

```swift
if iphone.batterieFaible {
    print("Batterie faible")
}
```

6. Attention : pas de propriété stockée

Une extension ne peut pas ajouter directement une nouvelle propriété stockée.

Ceci n'est pas permis :

```swift
extension Appareil {
    var couleur: String = "Noir"
}
```

Mais une propriété calculée est permise :

```swift
extension Appareil {
    var descriptionBatterie: String {
        return "\(batterie)%"
    }
}
```

7. Étendre Int

On peut également étendre les types standards de Swift.

Exemple :

```swift
extension Int {
    var estPair: Bool {
        return self % 2 == 0
    }
}
```

Utilisation :

```swift
let nombre = 10
print(nombre.estPair)
```

Résultat :

```
Console
true
```

8. Étendre Double

```
extension Double {
    var pourcentage: String {
        return "\(self)%"
    }
}
```

Utilisation :

```
let batterie = 75.0
print(batterie.pourcentage)
```

Résultat :

```
Console
75.0%
```

9. Étendre String

```
extension String {
    var estVideOuEspaces: Bool {
        return self
            .trimmingCharacters(
                in: .whitespaces
            )
            .isEmpty
    }
}
```

Utilisation :

```
let nom = "  "
if nom.estVideOuEspaces {
    print("Nom invalide")
}
```

10. Ajouter plusieurs méthodes

```
extension String {
    func afficher() {
        print(self)
    }

    func afficherMajuscules() {
        print(self.uppercased())
    }
}
```

Utilisation :

```
let message = "Bonjour"
message.afficher()
message.afficherMajuscules()
```

11. Extensions et protocoles

Une extension peut servir à faire adopter un protocole à un type.

Supposons :

```
protocol Affichable {
    func afficher()
}
```

Puis :

```
class Notification {
    var message: String

    init(message: String) {
        self.message = message
    }
}
```

On peut ajouter la conformité séparément :

```
extension Notification: Affichable {
    func afficher() {
        print(message)
    }
}
```

12. Pourquoi séparer la conformité ?

Cela améliore souvent l'organisation.

Au lieu de :

```
class Notification:
    Affichable,
    Connectable,
    Rechargeable {
```

on peut séparer :

```
class Notification {
    // propriétés principales
}
```

Puis :

```swift
extension Notification: Affichable {
    // comportement Affichable
}
```

13. Extension d'un protocole

Swift permet également d'ajouter une implémentation par défaut à un protocole.

Exemple :

```swift
protocol Identifiable {
    var nom: String { get }
}
```

Puis :

```swift
extension Identifiable {
    func afficherNom() {
        print("Nom : \(nom)")
    }
}
```

14. Utiliser l'implémentation par défaut

```swift
struct Utilisateur: Identifiable {
    var nom: String
}
```

Puis :

```swift
let utilisateur =
    Utilisateur(nom: "Alex")
utilisateur.afficherNom()
```

Même si Utilisateur ne définit pas directement afficherNom(), il reçoit l'implémentation fournie par l'extension du protocole.

15. Organisation du code avec les extensions

Une classe peut être organisée par responsabilités. Reprenons Appareil et ses deux extensions précédentes :

```swift
class Appareil { ... }

extension Appareil {
    func afficherEtat() { ... }
}

extension Appareil {
    var batterieFaible: Bool { ... }
}
```

Le même type est réparti en plusieurs blocs logiques.

16. Exemple — Position GPS avec extensions

```
struct PositionGPS {
    var latitude: Double
    var longitude: Double
}
```

Extension :

```
extension PositionGPS {
    func afficher() {
        print(
            "Lat: \(latitude), Lon: \(longitude)"
        )
    }

    var description: String {
        return "\(latitude), \(longitude)"
    }
}
```

Utilisation :

```
let position = PositionGPS(
    latitude: 45.50,
    longitude: -73.56
)
position.afficher()
```

17. Exercice — Extension de Batterie

Créer :

```
struct Batterie {
    var niveau: Int
}
```

Ajouter une extension contenant :

```
var estFaible: Bool
```

qui retourne true si :

```
niveau <= 20
```

Ajouter aussi :

```
func afficherEtat()
```

qui affiche :

```
Console
Batterie : 75%
```

18. Exercice — Validation d'un courriel

Créer une extension de :

```
String
```

avec :

```
var estCourrielValide: Bool
```

Pour simplifier, considérer un courriel valide s'il contient :

```
@
.
```

Exemple :

```
let courriel = "test@test.com"
print(courriel.estCourrielValide)
```

19. À retenir

Une extension se déclare avec :

```
extension Type {
}
```

Elle permet d'ajouter :

- Méthodes
- Propriétés calculées
- Conformité à des protocoles
- Implémentations par défaut

Elle ne permet pas d'ajouter directement une propriété stockée.

20. Vision globale

```
Type existant
     │
     ↓
  extension
     │
     ├── méthodes
     ├── propriétés calculées
     ├── protocoles
     └── organisation du code
```